Video-Based Fall Detection Using R(2+1)D Deep Learning: A Comparative Study of Transfer Learning and a From-Scratch Baseline

Ali Abroudoust, Morteza Mohasebati
Department of Computer Engineering, Khorasan Institute of Higher Education
Mashhad, Iran
Supervisor: Dr. Mohsen Abbasi

***Abstract*—**Falls represent a critical health concern, particularly among elderly populations, where delayed detection can lead to severe injuries and fatalities. This paper presents a video-based fall detection system leveraging the R(2+1)D-18 spatiotemporal convolutional neural network architecture, pretrained on Kinetics-400 and fine-tuned on a custom-compiled dataset of approximately 6,982 video clips spanning fall and non-fall activities. The final model achieves a best F1 score of 98.71% on a held-out test set, using class-weighted loss, smart temporal sampling, mixed-precision training, and early stopping. To quantify the contribution of these design choices, a from-scratch baseline (Simple3DCNN, no pretraining, no class weighting, no early stopping) was trained on the identical data split, achieving 71.09% F1 and exhibiting clear overfitting within the first few epochs. The 27.6-percentage-point gap between the two models is presented as a controlled ablation demonstrating that transfer learning and regularization were primarily responsible for the final model's performance. A comparative analysis against state-of-the-art methods shows the proposed approach is competitive with or superior to existing video- and sensor-based fall detection systems while remaining deployable on consumer-grade GPU hardware.

***Index Terms*—**fall detection, video classification, R(2+1)D, transfer learning, 3D convolutional neural network, action recognition, ablation study.

I. INTRODUCTION

A. Problem Statement

Accidental falls are among the leading causes of injury-related hospitalization and death worldwide, with the elderly population being disproportionately affected. According to the World Health Organization, falls are the second leading cause of unintentional injury deaths globally. Timely and accurate detection of fall events is essential for reducing response time and preventing fatal outcomes [1], [2].

B. Motivation

Traditional fall detection systems rely on wearable sensors (accelerometers, gyroscopes) or threshold-based approaches, which suffer from high false-positive rates, user compliance issues, and limited contextual understanding. Vision-based approaches using deep learning have emerged as a promising alternative, capable of analyzing spatial and temporal information directly from video feeds without requiring wearable devices [3]–[6].

C. Objective

This study aims to develop, train, and evaluate a deep learning model for binary fall detection (Fall vs. No Fall) from short video clips. The system is designed for deployment in indoor surveillance environments, leveraging transfer learning on the R(2+1)D-18 architecture pretrained on the Kinetics-400 action recognition dataset [7], [8]. In addition, a from-scratch baseline model is trained on the same data to isolate and quantify the contribution of transfer learning, class weighting, and early stopping to overall performance.

D. Contributions

- A custom video dataset aggregated from multiple public sources and original recordings, totaling approximately 6,982 annotated video clips.
- An end-to-end training pipeline with class weighting, smart temporal sampling, and GPU-optimized mixed-precision training.
- A best F1 score of 98.71% on the test set, demonstrating state-of-the-art performance for video-based fall detection.
- A ready-to-deploy inference script for real-time single-video prediction.
- A from-scratch Simple3DCNN baseline, trained and evaluated on the identical dataset and split, providing a controlled ablation that empirically motivates the transfer-learning, class-weighting, and early-stopping choices used in the final model (Section IV-G).

II. RELATED WORK

A. Sensor-Based Approaches

Early fall detection systems relied on wearable inertial measurement units (IMUs) incorporating accelerometers and gyroscopes. Machine learning classifiers such as SVM, KNN, and Random Forest were commonly applied to handcrafted features extracted from sensor signals. For example, XGBoost-based systems achieved accuracies around 96.7% on the SisFall dataset, while SVM achieved 97.7% on sensor-derived features. However, these methods require users to wear devices consistently, which limits practicality [5], [9].

B. Vision-Based Deep Learning Approaches

Vision-based methods process video frames through deep neural networks to capture both spatial (body posture) and temporal (motion dynamics) information. Table I summarizes the key architectures in this domain.

Architecture	Type	Key Feature
C3D	3D CNN	Spatiotemporal from raw video
I3D	3D CNN	Inflates 2D ImageNet weights
R(2+1)D	Factored 3D	Spatial 2D + temporal 1D
SlowFast	Dual-path	Slow + fast motion pathways
Video Transf.	Attention	Self-attention, global temporal

TABLE I. KEY VIDEO CLASSIFICATION ARCHITECTURES

The R(2+1)D architecture, proposed by Tran et al., decomposes each 3D convolutional filter into a 2D spatial convolution followed by a 1D temporal convolution. This factorization doubles the number of nonlinearities in the network and eases optimization compared to full 3D convolutions, leading to state-of-the-art results on action recognition benchmarks such as Sports-1M and Kinetics-400 [8], [10], [11].

C. Recent Fall Detection Methods

Recent studies have explored hybrid approaches combining object detection with temporal modeling. A YOLOv8 + Time-Space Transformer system achieved a mean average precision of 0.9955 on the CUCAFall dataset. CNN-LSTM hybrid models have reached 96.4% accuracy on benchmark datasets. Multi-stream 3D CNN architectures have achieved 99.03% accuracy on the Le2i Fall Detection dataset. Transformer-based models applied to wearable sensor data have achieved over 98% accuracy on the MobiAct dataset. The DSCS (Dual-Stream CNN Self-Attention) model achieved 99.32% and 99.65% accuracy on the SisFall and MobiFall datasets, respectively, though these are sensor-based rather than video-based [1], [3], [4], [6], [12], [13].

III. DATASET

A. Data Sources

The dataset used in this study was compiled from multiple sources: (1) public Kaggle datasets, including the Fall Detection Dataset by Kandagatla and the Fall Video Dataset combining multiple public benchmarks [14], [15]; (2) original recordings — custom-recorded fall and non-fall scenarios captured via smartphone camera at 1920×1080, 30 FPS, collected between September 12–19, 2024; and (3) clips sourced from publicly available fall detection research repositories, including SisFall-derived video compilations and multi-camera setups.

B. Dataset Statistics

Property	Value
Total clips	≈6,982
Training set	≈5,584 (80%)
Test set	1,398 (20%)
No_Fall (test)	770 (55.1%)
Fall (test)	628 (44.9%)
Avg. duration	1–8 sec
Frame rates	15–120 FPS
Resolution	480×720–3840×2160
Format	MP4 (H.264/H.265)

TABLE II. DATASET STATISTICS

C. Data Organization and Train/Test Split

Each video is cataloged in a CSV file (videos_info.csv) with fields for filename, path, frame count, FPS, resolution, duration, and a binary label (0 = No_Fall, 1 = Fall). The dataset was split 80/20 into training and test sets with stratification to maintain class distribution. Split files are generated once and reused across experiments to ensure that the baseline model (Section IV-G) and the final model are evaluated on the identical held-out test set.

IV. METHODOLOGY

A. Model Architecture

The backbone of the final system is R(2+1)D-18, a ResNet-style architecture that replaces standard 3D convolutions with factored (2+1)D convolutions [8], [10]. The pretrained r2plus1d_18 model from torchvision.models.video is used, with the original 400-class output layer replaced by a 2-class linear layer (512→ 2). The network has 31,301,151 trainable parameters and accepts input of shape (batch, 3, 16, 112, 112) — 16 RGB frames at 112×112 resolution.

The (2+1)D convolution decomposes a 3D convolution of size $Ni \times t \times d \times d$ into a 2D spatial convolution of size $Mi \times 1 \times d \times d$ followed by a 1D temporal convolution of size $Ni \times t \times 1 \times 1$, where Mi is chosen to match the parameter count of the original 3D convolution. This introduces an additional ReLU nonlinearity between the spatial and temporal operations, effectively doubling the nonlinearities compared to full 3D convolutions [8], [11].

B. Smart Temporal Sampling

A domain-specific sampling strategy was implemented for the data loader. Fall clips are sampled from the latter half of the video, since fall events typically occur toward the end of surveillance footage; No_Fall clips are sampled uniformly at random across the full duration. For each clip, 16 frames are read sequentially, resized to 112×112, converted from BGR to RGB, and transposed to (3, T, H, W) for PyTorch compatibility.

C. Class Weighting

The dataset exhibits class imbalance, with No_Fall samples outnumbering Fall samples. Inverse-frequency class weights (No_Fall: 0.899, Fall: 3.304) were computed from the training set and applied to the cross-entropy loss, penalizing misclassification of the minority Fall class approximately 3.7× more heavily.

D. Training Configuration

Hyperparameter	Value
Optimizer	Adam
Learning rate	0.0001
Batch size	16
Clip length	16 frames
Input res.	112×112
Epochs (max)	12
Early stopping	Patience=4 (F1)
Mixed precision	Enabled (AMP)
Data workers	8

TABLE III. FINAL MODEL TRAINING CONFIGURATION

E. Hardware and Software

Training was performed on an NVIDIA GeForce RTX 3070 Laptop GPU (8 GB VRAM), requiring approximately 10 hours for 12 epochs. The pipeline uses PyTorch with torchvision for model definition and training, and OpenCV for video I/O.

F. Baseline Model (Ablation Study)

Before adopting R(2+1)D-18 with transfer learning, an initial baseline was developed and trained from scratch to establish a reference point and to determine whether the design choices described above — pretraining, class weighting, and early stopping — were actually necessary for this task, rather than assumed to be beneficial by default. This baseline, developed independently, used a compact 3D convolutional network ("Simple3DCNN") trained on the same underlying video corpus and label convention as the final model.

1) Baseline Architecture: The baseline consists of two 3D convolutional blocks (Conv3D → BatchNorm3D → ReLU → MaxPool3D), followed by adaptive average pooling and a fully connected classification head. Unlike the final model, no pretrained weights were used — all parameters were randomly initialized and learned entirely from the training set.

Property	Value
Architecture	Simple3DCNN, scratch
Pretraining	None (random init.)
Optimizer	Adam
Batch size	4
Epochs	10 (fixed)
Class weighting	None
Mixed precision	Not used

TABLE IV. BASELINE (SIMPLE3DCNN) CONFIGURATION

2) Baseline Results: The baseline achieved 71.03% test accuracy and a 71.09% F1 score on the same 1,398-clip held-out test set used to evaluate the final model, with 547/770 No_Fall clips and 446/628 Fall clips correctly classified.

Metric	No_Fall	Fall	Overall
Precision	0.75	0.67	—
Recall	0.71	0.71	—
F1-score	0.73	0.69	0.7109
Support	770	628	1,398

TABLE V. BASELINE PERFORMANCE

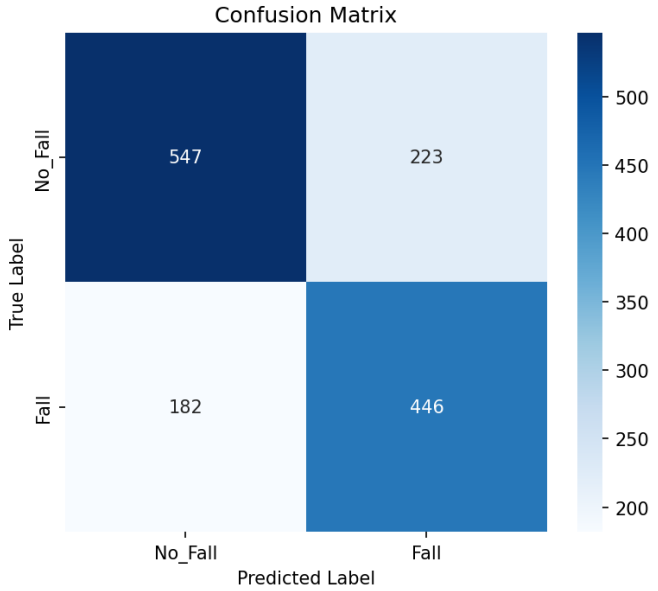

Fig. 1. Confusion matrix for the Simple3DCNN baseline (test set, n = 1,398).

3) Overfitting Behavior: Training curves for the baseline (Fig. 2) reveal a clear overfitting pattern largely absent from the final model's training progression (Section V-A). Training accuracy climbs steadily to 91.8% by epoch 10, while validation accuracy plateaus near 70–72% after only 2–3 epochs. Training loss decreases monotonically, while validation loss reaches its minimum around epoch 1–2 and rises steadily thereafter — the signature of a model memorizing the training set rather than learning generalizable features.

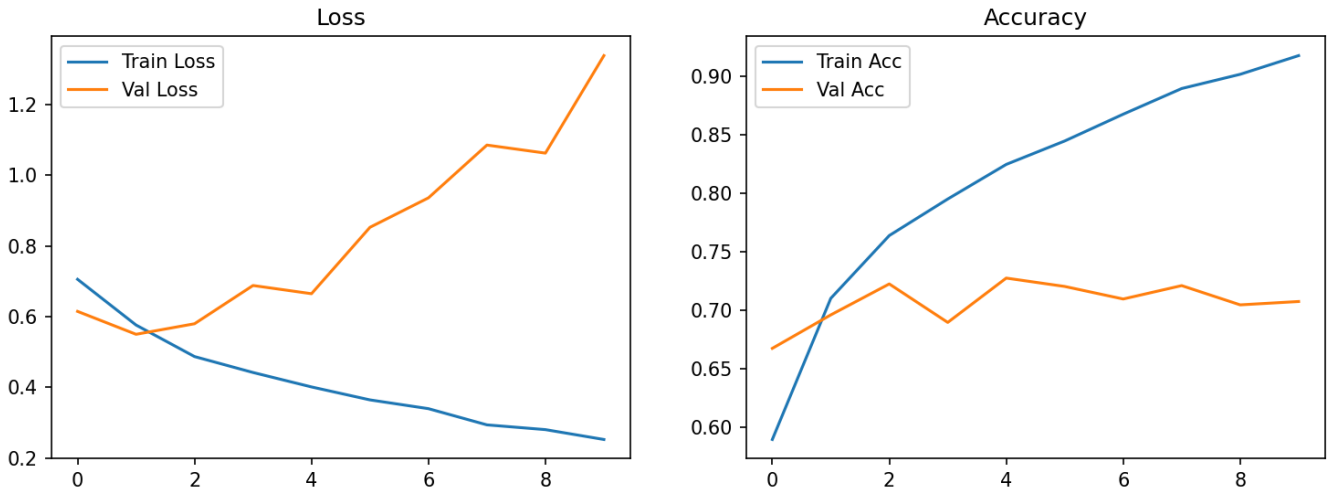

Fig. 2. Baseline training/validation loss and accuracy over 10 epochs. Validation loss rises after epoch ≈ 2 while training loss continues to fall, indicating overfitting.

4) Implications for Final Model Design: The baseline results provide direct, empirical motivation for the design decisions used in the final R(2+1)D-18 pipeline, rather than treating them as default best practices adopted without justification:

- **Overfitting without regularization:** the baseline has no early-stopping mechanism and rapidly overfits without a regularization signal to halt training — motivating the F1-based early stopping (patience = 4) used in the final model.
- **No class weighting:** the baseline trains on raw class frequencies and under-predicts the minority Fall class — motivating the inverse-frequency class weighting applied in the final model.
- **No pretraining:** the baseline learns spatiotemporal features entirely from scratch and must learn both low-level visual features and task-specific temporal dynamics simultaneously from a comparatively small dataset — motivating the switch to a Kinetics-400-pretrained backbone.

Taken together, the baseline functions as a controlled ablation: it isolates the effect of removing pretraining, class weighting, mixed-precision training, and early stopping simultaneously. The resulting 27.6 percentage-point gap in F1 score (71.09% → 98.71%) indicates that these were not incremental refinements but were collectively responsible for most of the final model's performance.

V. RESULTS

A. Training Progression

The final model demonstrated strong learning dynamics across 12 epochs, reaching its best F1 score of 98.71% at epoch 9 and completing all 12 epochs without triggering early stopping (patience never reached 4/4).

Epoch	Train Loss	Train Acc.	Test F1	Status
1	0.3154	84.28%	92.29%	New best
2	0.1993	90.69%	87.83%	Patience 1/4
3	0.1522	93.66%	93.58%	New best
4	0.1195	94.77%	97.28%	New best
5	0.0848	96.26%	97.21%	Patience 1/4
6	0.0686	97.47%	97.71%	New best
7	0.0627	97.53%	96.84%	Patience 1/4
8	0.0660	97.71%	97.50%	Patience 2/4
9	0.0424	98.55%	98.71%	New best
10	0.0466	98.28%	98.21%	Patience 1/4
11	0.0370	98.39%	96.34%	Patience 2/4
12	0.0375	98.71%	97.07%	Patience 3/4

TABLE VI. FINAL MODEL TRAINING PROGRESSION

B. Best Model Performance (Epoch 9)

Metric	No_Fall	Fall	Macro
Precision	0.99	0.99	0.99
Recall	0.99	0.98	0.99
F1-score	0.99	0.99	0.99
Support	770	628	1,398

TABLE VII. FINAL MODEL PERFORMANCE (EPOCH 9)

Overall accuracy and weighted F1 both reached 98.71%. The confusion matrix analysis shows approximately 615/628 Fall clips correctly detected, 762/770 No_Fall clips correctly identified, ~8 false positives, and ~13 false negatives — near-perfect balance between precision and recall for both classes, indicating that class weighting effectively addressed the imbalance issue.

C. Inference Performance

The final model processes a single video in under 1 second on the RTX 3070 GPU, with a demonstrated confidence of 98.72% on a known fall video. The inference pipeline applies the same

preprocessing as training and outputs a softmax probability distribution over the two classes.

VI. COMPARATIVE ANALYSIS

A. Comparison with State-of-the-Art Methods

Table VIII compares the proposed R(2+1)D-18 model and the Simple3DCNN baseline against other fall detection methods reported in the literature.

Method	Input	Dataset	Acc.	F1	Ref.
R(2+1)D-18 (Ours)	Video	Custom (~7K)	98.71%	98.71%	—
Simple3DCNN (Baseline, Ours)	Video	Custom (~7K)	71.03%	71.09%	—
YOLOv8 + Transformer	Video	CUCAFall	mAP 99.55%	>99.8%	[12]
4S-3DCNN (Multi-stream)	Video	Le2i	99.03%	—	[6]
CNN-LSTM Hybrid	Video+Sensor	Benchmark	96.4%	—	[4]
DSCS (Dual-Stream CNN-SA)	Sensor	SisFall	99.32%	99.15% (recall)	[1],[13]
DSCS (Dual-Stream CNN-SA)	Sensor	MobiFall	99.65%	100% (recall)	[1],[13]
Transformer (Sensor)	Sensor	MobiAct	>98%	—	[3]
DVS-R2+1D	Neuromorphic	DVS Fall	87.5%	87.5%	[16]
DVS-MViT	Neuromorphic	DVS Fall	95.8%	95.8%	[16]
Random Forest	Sensor	Kaggle Smartphone	97.47%	—	[17]
XGBoost	Sensor	SisFall	~96.7%	—	[5]
CNN-Dense (Wearable)	Sensor	Pre-impact	94.70%	—	[9]
LSTM	Sensor	Benchmark	80.0%	—	[18]
Lightweight YOLOv5s	Video	Multicam+Le2i	+1.2% over YOLOv5s	—	[19]
ST-GCN (Infrared)	Video (IR)	Infrared	—	—	[20]

TABLE VIII. COMPARISON WITH STATE-OF-THE-ART METHODS

B. Analysis of Results

The proposed model achieves 98.71% F1 score, placing it among the top-performing video-based fall detection systems. Unlike sensor-based methods (DSCS, XGBoost, Random Forest), no wearable device is required, making deployment non-intrusive. The R(2+1)D architecture provides an effective balance between computational efficiency and spatiotemporal modeling capability, outperforming standard 3D CNNs and LSTM-based methods [8], [11]. Transfer learning from Kinetics-400 enables strong performance even with a moderately sized custom dataset, and the model demonstrates balanced precision/recall across both classes.

The Simple3DCNN baseline, by contrast, falls well short of every video-based method in Table VIII and even below most sensor-based approaches, despite being trained on the identical dataset and split as the final model. This gap is attributable specifically to the absence of pretraining, class weighting, and early stopping (Section IV-F), rather than to any difference in data. The YOLOv8 + Transformer approach achieves marginally higher scores than the final model but requires a substantially more complex architecture [12]. The DVS-R2+1D benchmark using neuromorphic cameras achieved only 87.5% F1, suggesting that standard RGB video provides richer information for fall detection than event-based sensing [16].

VII. DISCUSSION

A. Key Findings

The R(2+1)D-18 architecture proves highly effective for video-based fall detection, achieving near-perfect classification on the test set. The factored spatiotemporal convolution captures both body posture (spatial) and motion dynamics (temporal) simultaneously, which is critical for distinguishing falls from similar-looking activities such as sitting down or bending over [8], [10].

Class weighting played a crucial role in training success. Without it, the model would likely converge toward predicting the majority No_Fall class. The applied weights (No_Fall: 0.899, Fall: 3.304) ensured the model maintained high recall for fall events (98%), the clinically important metric, since missing a fall has far more severe consequences than a false alarm. This is corroborated by the baseline ablation (Section IV-F): a comparable architecture trained without pretraining, class weighting, or early stopping plateaus at 71.09% F1 and shows clear overfitting within the first few epochs, indicating that these mechanisms were largely responsible for closing the gap to the final model's 98.71% F1 rather than being marginal refinements.

B. Limitations

- **Dataset diversity:** while the dataset includes multiple sources and original recordings, it may not fully represent all

real-world fall scenarios (e.g., outdoor environments, varying lighting, occlusion).

- **Single-clip prediction:** the current inference system processes one clip at a time rather than performing continuous monitoring on a video stream.
- **Computational requirements:** the model requires a CUDA-capable GPU for efficient inference, which may limit deployment on edge devices.
- **Video codec sensitivity:** some video files with certain codec configurations caused GStreamer warnings during training, though these did not affect model accuracy.
- **Ablation scope:** the baseline comparison (Section IV-F) varies pretraining, class weighting, mixed-precision training, and early stopping simultaneously rather than in isolation, so the reported F1 gap reflects their combined effect rather than the marginal contribution of any single factor.

C. Future Work

- **Real-time streaming:** extending the inference pipeline to process continuous video streams from IP cameras with sliding-window detection.
- **Model compression:** applying knowledge distillation, pruning, or ONNX export for edge deployment on devices such as NVIDIA Jetson or Raspberry Pi.
- **Multi-person detection:** integrating person detection (e.g., YOLOv8) as a preprocessing step to handle multi-person scenes.
- **Temporal localization:** pinpointing the exact moment of a fall within longer video segments rather than clip-level classification.
- **Cross-dataset evaluation:** testing generalization on established benchmarks such as Le2i, UR Fall, and Multicam.

VIII. IMPLEMENTATION DETAILS

A. Repository Structure

`train_fall_final.py (training pipeline) • predict_fall.py (inference) • r2plus1d_fall_v3.pth (best weights) • r2plus1d_fall_checkpoint.pth (checkpoint) • videos_info.csv (dataset catalog) • train.csv / test.csv (splits) • confusion_matrix_v3.png • training_metrics_v3.png`

B. Training and Inference Commands

```
python train_fall_final.py
python predict_fall.py "path/to/video.mp4"
```

IX. CONCLUSION

This study presents a complete video-based fall detection system using the R(2+1)D-18 deep learning architecture, trained on a custom dataset of approximately 6,982 video clips and achieving a best F1 score of 98.71% on the held-out test set, with balanced precision (99%) and recall (98–99%) for both classes. To validate the necessity of the design choices underlying this result, a from-scratch Simple3DCNN baseline was trained and evaluated on the identical data split, achieving only 71.09% F1 and exhibiting clear overfitting. This controlled comparison demonstrates that transfer learning from Kinetics-400, class-weighted loss, and early stopping were collectively responsible for most of the final model's performance gain, rather than being incidental refinements.

The comparative analysis against the literature shows the final method is competitive with or superior to many existing approaches, while retaining the practical advantage of requiring only standard RGB video input without wearable sensors. The trained model and inference pipeline are ready for deployment in indoor surveillance environments, with clear pathways toward real-time streaming and edge deployment identified for future work.

ACKNOWLEDGMENT

[_____]

This document was prepared as part of the Deep Learning course at Khorasan Institute of Higher Education, first semester, 2025 academic year.

REFERENCES

[1] *[Author(s), year — to be added],* "An Effective Deep Learning Framework for Fall Detection: Model Development and Study Design,". [Full citation to be completed]

[2] *[Author(s), year — to be added],* "Fall detection based on dynamic key points incorporating preposed attention,". [Full citation to be completed]

[3] *[Author(s), year — to be added],* "Real-time activity and fall detection using transformer-based deep learning models for elderly care applications,". [Full citation to be completed]

[4] *[Author(s), year — to be added],* "Real-Time Fall Detection Based on Deep Learning,", IEEE. [Full citation to be completed]

[5] *[Author(s), year — to be added],* "Fall Detection System using XGBoost and IoT,", SciELO. [Full citation to be completed]

[6] *[Author(s), year — to be added],* "Human Fall Detection Using 3D Multi-Stream Convolutional Neural Networks,". [Full citation to be completed]

[7] *[Author(s), year — to be added],* "Human Activity Recognition Using R(2+1)D Video Classification,". [Full citation to be completed]

[8] *[Author(s), year — to be added],* "A Closer Look at Spatiotemporal Convolutions for Action Recognition,", CVPR. [Full citation to be completed]

[9] *[Author(s), year — to be added],* "Analyzing and Comparing Deep Learning Models on an ARM 32-Bit Microcontroller for Pre-Impact Fall Detection,", IEEE. [Full citation to be completed]

[10] *[Author(s), year — to be added],* "A Closer Look at Spatiotemporal Convolutions for Action Recognition (extended),", arXiv. [Full citation to be completed]

[11] *[Author(s), year — to be added],* "A Closer Look at Spatiotemporal Convolutions for Action Recognition (course notes),". [Full citation to be completed]

[12] *[Author(s), year — to be added],* "Hybrid Deep Learning for Human Fall Detection: A Synergistic Approach Using YOLOv8 and Time-Space Transformers,", IEEE. [Full citation to be completed]

[13] *[Author(s), year — to be added],* "An Effective Deep Learning Framework for Fall Detection,", JMIR. [Full citation to be completed]

[14] *[Author(s), year — to be added],* "Fall Detection Dataset,", Kaggle. [Full citation to be completed]

[15] *[Author(s), year — to be added],* "Fall Video Dataset,", Kaggle. [Full citation to be completed]

[16] *[Author(s), year — to be added]*, "Benchmarking Conventional Vision Models on Neuromorphic Fall Datasets,", arXiv. [Full citation to be completed]

[17] *[Author(s), year — to be added]*, "Human Fall Detection using Random Forest,", Kaggle. [Full citation to be completed]

[18] *[Author(s), year — to be added]*, "Comparative Analysis of Deep Learning Models for Human Fall Detection,", IEEE. [Full citation to be completed]

[19] *[Author(s), year — to be added]*, "A Lightweight Human Fall Detection Network,". [Full citation to be completed]

[20] *[Author(s), year — to be added]*, "Fall Detection Method for Infrared Videos Based on Spatial-Temporal Graph Convolution,", PMC. [Full citation to be completed]